

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Muslima Kosmagambetova
Student Identification Number	230103269
Date & Laboratory Session	17.09.2026
Workstation / Laptop Make & Model	Macbook Air 13, Apple M4, 16GB
Processor SKU / SoC Model	Apple M4
Core Topology (Physical P-cores / E-cores)	Physical Cores: 10 (4 P-cores + 6 E-cores) Logical SMT Threads: 10 (no SMT on Apple Silicon)
Cache Hierarchy Capacity	L1: 128 KB L2: 64 MB L3: N/A MB
Host OS & Python Runtime	e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

Worker Cores (p) Kernel Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	0.0667	1.98x	1.00x	100.0%
p = 2	0.0717	1.84x	2.00x	92
p = 4	0.0809	1.63x	4.00x	40
p = 8	0.0939	1.40x	8.00x	17.5
p = 12	0.1894	0.70x	12.00x	5.8
p = 16	0.1907	0.69x	16.00x	4.3

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

I looked at my results and the speedup never really grows. Even at $p = 2$ the time is already worse than the baseline (0.0717s vs 0.0667s), so my inflection point is $p^* = 1$. After that every extra worker only makes the program slower, and the biggest drop is between $p = 8$ (0.0939s) and $p = 12$ (0.1894s), where the time becomes two times worse.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

$$S(4) = T(1) / T(4) = 0.0667 / 0.0809 = 0.824$$

$$S(4) = 1 / [(1 - P) + P/4]$$

$$0.824 = 1 / [(1 - P) + P/4]$$

$$(1 - P) + P/4 = 1 / 0.824 = 1.2136$$

$$1 - P + 0.25P = 1.2136$$

$$1 - 0.75P = 1.2136$$

$$-0.75P = 0.2136$$

$$P = -0.285$$

The result is negative, and a parallel fraction cannot be negative, because P must be between 0 and 1. This means that Amdahl's Law does not fit my measurement at all. Amdahl's Law assumes that parallel work is free and only the serial part costs time, but in my experiment the process creation and communication cost more than the parallel work saves. So for my silicon the practical value is $P \approx 0$ (no measurable parallel fraction), and the negative number just shows how big the overhead is.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

$$S_{\max} = 1 / (1 - P)$$

With $P \approx 0$, $S_{\max} = 1 / (1 - 0) = 1.0x$. If I use my formal negative value $P = -0.285$, I get $S_{\max} = 1 / 1.285 = 0.78x$, which is even less than one.

This means that for this exact workload the theoretical limit is no speedup at all. If my department bought a 128-core server node, this program would not run 128 times faster, and probably not even 2 times faster. It could even run slower, because more processes mean more spawn overhead and more scheduling work for the OS. The lesson for me is that buying hardware does not fix a program: first the workload must be big enough per worker, and the overhead must be small compared to the useful work. Only then more cores help.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting

the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```
import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")
```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.8258s	0.8102s	0.8021s	0,8124s	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.8098s	0.8133s	0.8092s	0,810766s	

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

On a 64-bit system a pointer is 8 bytes (64 bits). A CPython list does not store the numbers themselves, it stores pointers to integer objects, so one list slot takes 8 bytes.

A cache line is 64 bytes, so one cache line can hold $64 / 8 = 8$ list slots. If two threads use neighbour indices like 0 and 1, the distance between them is only 8 bytes, so they are almost always inside the same 64-byte line.

With a stride of 16 the distance is $16 \times 8 = 128$ bytes. Because $128 > 64$, two padded indices can never be inside the same 64-byte line, even in the worst case when the array does not start exactly at a line boundary. In that bad case one line can still cover at most 64 bytes, and my elements are 128 bytes apart, so at least one full line is always between them.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Manual padding by hand is ugly and wastes memory, so real languages give special tools:

1. **C++11 alignas(64)**. I can write `struct alignas(64) Counter { long value; };` and the compiler puts every counter at the start of its own cache line. Since C++17 there is also `std::hardware_destructive_interference_size`, which gives the correct line size for the target CPU, so the code stays portable.
2. **Java @jdk.internal.vm.annotation.Contended**. I put this annotation on a field and the JVM adds padding around it at runtime. Java uses it internally, for example in `LongAdder`, which is exactly a counter designed to avoid contention.

A third option I found is Go, where people add a dummy array field like `_ [56]byte` inside a struct, or use per-goroutine values instead of shared ones.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```
class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")
```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	No	0.1152s	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.3066s	

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

In my run the unsafe counter was **not** corrupted: I got exactly 2,000,000 of 2,000,000, and zero lost updates. So I cannot show a corrupted number, but I can explain why corruption is possible and why it did not happen here.

The line `self.val += 1` is not one machine operation. It is three steps: **LOAD** the current value into a register, **ADD** one to it, and **STORE** the result back. If Thread A loads 100 and then the OS switches to Thread B, which also loads 100, adds one and stores 101, and after that Thread A stores its own 101, then two increments produced only one. This is a lost update, and with millions of operations the final sum becomes much smaller than the target.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:import time
import threading

TOTAL_OPS = 2_000_000
NUM_THREADS = 4

```
class LocklessCounter:
    def __init__(self, num_threads):
        self.partials = [0] * num_threads

    def run(self, total_ops, num_threads):
        ops_per_thread = total_ops // num_threads

        def work(tid):
            local = 0          # private variable of this thread
            for _ in range(ops_per_thread):
                local += 1
            self.partials[tid] = local # one write at the end

        threads = [
            threading.Thread(target=work, args=(i,))
            for i in range(num_threads)
        ]
        start = time.perf_counter()
        for t in threads:
            t.start()
        for t in threads:
            t.join()
        total = sum(self.partials) # reduction stage
        return total, time.perf_counter() - start

if __name__ == "__main__":
    counter = LocklessCounter(NUM_THREADS)
    value, elapsed = counter.run(TOTAL_OPS, NUM_THREADS)
    print(f"Lockless: Value = {value:}, / {TOTAL_OPS:} | Time: {elapsed:.4f}s")
```

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

I think the difference is about the problem itself, not about the tool. When I optimize a lock, for example with an adaptive spinlock or a read-write lock, I only make the shared resource a little cheaper to take. But the resource is still shared, so threads still wait for each other, and the critical section is still serial. By Amdahl's Law this serial part sets a hard limit on my speedup, no matter how good my lock is.

When I remove shared mutable state, the problem disappears. Each thread has its own data, so there is no waiting, no OS call, no cache line moving between cores. The cost of combining results is paid only once at the end, and it does not grow with the number of operations, only with the number of threads.

There is also a hardware reason. A lock variable is one memory address, and every core that tries to take it writes to the same cache line, so the line moves between caches all the time. With private buffers each core works with its own line and the caches stay quiet. This is why in high-throughput systems people prefer patterns like sharding, per-thread accumulators or message passing instead of smarter locks.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queuing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()
```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	1.1167s	1.00x	0.4507s	1.00x
w = 2 Workers	1.0729s	0.96x	0.6542s	1.45x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 4 Workers	1.4088s	1.26x	1.3406s	2.97x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

In the compute-bound suite my times were 1.1167s, 1.0729s and 1.4088s for 1, 2 and 4 workers. Four workers did four times more total work in almost the same time, so the scaling factor is only 1.26x. This is good parallel behaviour: each process does its math inside CPU registers, needs almost no data from memory, and my M4 has enough physical cores to run four of them at the same time. The cores are independent, so they do not disturb each other.

In the memory-bound suite the times were 0.4507s, 0.6542s and 1.3406s, so four workers are 2.97x slower than one. Here every process streams a big array of about 400 MB from main memory and does almost no math. The cores are not the bottleneck, the memory bus is. All cores share the same memory controller and the same DRAM bandwidth, so when four processes stream data at the same time, they fight for the same channel. Each one gets only a part of the bandwidth, and requests wait in a queue.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

My laptop is a MacBook Air 13 with Apple M4 and 16 GB of unified memory. Apple Silicon does not use normal DIMM modules, it uses LPDDR5X memory soldered next to the SoC in a unified memory architecture, where CPU, GPU and Neural Engine share the same memory pool. For the base M4 the memory bandwidth is about **120 GB/s**.

This number is a hard physical limit for all cores together, not for each core. If one process can already use, for example, 60 GB/s, then two processes already reach the limit and the third and fourth get nothing extra. The bus width and the number of memory channels decide how many bytes per second can move at all, and no scheduler can change this. That is exactly what I see in my table: from one to two workers the time grows 1.45x, and from one to four workers 2.97x, which is close to linear growth, as if the work became fully serial on the memory side.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature

vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

If my embedding pipeline is memory-bandwidth saturated, then CPU OpenMP pragmas will not help much. OpenMP only gives me more threads on the same CPU, and those threads share the same memory controller that is already full. I would just add more waiting.

A GPU is the better choice in this case, but not because it has more cores. The real reason is that GPU memory has much higher bandwidth: modern HBM memory gives around 1–3 TB/s, which is ten or twenty times more than my laptop. Embedding transforms have low arithmetic intensity (few operations per byte), so the winner is the device that can move bytes faster.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' div="" uploaded<=""></id>.txt'>	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot $T \times S$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while

- multiprocessing scales? (Forces them to articulate the GIL).
- o Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? o
 - o Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.